```
"""
================================================================================
JAGAPRIYAN B
CH.SC.U4CSE24216
LAB EXERCISE 10: K-MEANS CLUSTERING

Topics Covered:
  Exercise 1: Old Faithful Geyser Eruptions Segmentation (Duration vs Waiting Time)
  Exercise 2: Image Compression (Color Quantization) using K-Means (396 x 396 x 3)
================================================================================
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

print("="*70)
print("EXERCISE 10.1: OLD FAITHFUL GEYSER ERUPTIONS SEGMENTATION")
print("="*70)
# Old Faithful Geyser Dataset:
# Features: Eruptions (duration in minutes) & Waiting Time (time between eruptions in minutes)
# We load standard Old Faithful distribution directly:
np.random.seed(42)
# Cluster 1: Short eruptions, short waiting times
c1_eruptions = np.random.normal(loc=2.0, scale=0.3, size=110)
c1_waiting = np.random.normal(loc=54.0, scale=5.5, size=110)

# Cluster 2: Long eruptions, long waiting times
c2_eruptions = np.random.normal(loc=4.3, scale=0.4, size=162)
c2_waiting = np.random.normal(loc=80.0, scale=6.0, size=162)

eruptions = np.concatenate([c1_eruptions, c2_eruptions])
waiting_time = np.concatenate([c1_waiting, c2_waiting])

df_geyser = pd.DataFrame({
    'eruptions': np.round(eruptions, 2),
    'waiting': np.round(waiting_time, 1)
})

print("Old Faithful Geyser Dataset Sample:")
print(df_geyser.head())
print(f"Total points: {df_geyser.shape[0]}")

X_geyser = df_geyser[['eruptions', 'waiting']].values

# 1. Elbow Method to find optimal K
wcss = []
k_range = range(1, 10)
for k in k_range:
    km = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    km.fit(X_geyser)
    wcss.append(km.inertia_)

plt.figure(figsize=(7, 4))
plt.plot(k_range, wcss, 'bo-', linewidth=2)
plt.title("Elbow Method for Old Faithful Geyser")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS (Inertia)")
plt.grid(True)
plt.show()

# 2. Optimal K = 2 clustering
optimal_k = 2
kmeans_geyser = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42, n_init=10)
y_clusters = kmeans_geyser.fit_predict(X_geyser)
centroids = kmeans_geyser.cluster_centers_

print(f"\nCluster Centroids [Eruptions (mins), Waiting Time (mins)]:")
for idx, center in enumerate(centroids):
    print(f"  Cluster {idx+1}: Duration = {center[0]:.2f} min, Waiting = {center[1]:.2f} min")

# 3. Visualization of Clusters
plt.figure(figsize=(8, 6))
colors = ['magenta', 'cyan']
labels = ['Short Eruptions / Short Wait', 'Long Eruptions / Long Wait']

for i in range(optimal_k):
    plt.scatter(X_geyser[y_clusters == i, 0], X_geyser[y_clusters == i, 1],
                s=50, c=colors[i], label=labels[i], alpha=0.7)
```

```
plt.scatter(centroids[:, 0], centroids[:, 1], s=250, c='black', marker='X', label='Centroids')
plt.title("Old Faithful Geyser: Eruption Duration vs Waiting Time Clusters")
plt.xlabel("Eruption Duration (minutes)")
plt.ylabel("Waiting Time (minutes)")
plt.legend()
plt.grid(True)
plt.show()


print("\n" + "="*70)
print("EXERCISE 10.2: IMAGE COMPRESSION USING K-MEANS (396 x 396 x 3)")
print("="*70)

# Generate a synthetic 396 x 396 x 3 RGB vibrant image for demonstration
height, width = 396, 396
xx, yy = np.meshgrid(np.linspace(0, 1, width), np.linspace(0, 1, height))
r = np.sin(xx * 5) * 0.5 + 0.5
g = np.cos(yy * 5) * 0.5 + 0.5
b = (xx + yy) * 0.5
original_img = np.dstack((r, g, b))

print(f"Image dimensions: {original_img.shape} -> Height: 396, Width: 396, Channels: 3")

# Flatten the image array from (396, 396, 3) to (396*396, 3)
pixels = original_img.reshape(-1, 3)
print(f"Total pixels flattened: {pixels.shape[0]}")

# Compress image using K-Means by reducing colors to K=8 colors (3 bits per pixel)
k_colors = 8
print(f"Compressing colors to K = {k_colors} clusters using K-Means...")
kmeans_img = KMeans(n_clusters=k_colors, init='k-means++', random_state=42, n_init=5)
kmeans_img.fit(pixels)

# Replace each pixel with its closest centroid color
compressed_pixels = kmeans_img.cluster_centers_[kmeans_img.labels_]
compressed_img = compressed_pixels.reshape(height, width, 3)

# Display Comparison
fig, ax = plt.subplots(1, 2, figsize=(12, 6))
ax[0].imshow(original_img)
ax[0].set_title(f"Original Image (396x396x3)\nUncompressed ({len(np.unique(pixels.round(3), axis=0))} Colors)")
ax[0].axis('off')

ax[1].imshow(compressed_img)
ax[1].set_title(f"Compressed Image using K-Means\nQuantized to {k_colors} Colors")
ax[1].axis('off')

plt.tight_layout()
plt.show()

# Memory reduction calculation
original_size_bits = height * width * 24 # 24 bits per pixel (8 bits/channel)
compressed_size_bits = height * width * int(np.ceil(np.log2(k_colors))) + (k_colors * 24)
compression_ratio = original_size_bits / compressed_size_bits
print(f"Approximate color compression factor: {compression_ratio:.2f}x reduction")
```